ML Algorithms — In-Depth Guide

Random Forest · XGBoost · LightGBM · Logistic Regression · Decision Trees
Gradient Descent · KMeans · KMeans++ · Bagging · Boosting · Gradient Boosting

---

This document explains the working of each major ML algorithm step-by-step — from the mathematical intuition to the exact algorithm, key hyperparameters, pros/cons, and common interview points. Ideal for MLP viva preparation.

1. Decision Tree

What is it?

A Decision Tree is a flowchart-like model that splits data recursively based on feature thresholds. Each internal node tests a feature, each branch represents an outcome, and each leaf node gives a prediction.

Algorithm — Step by Step

Step 1: Start with the entire dataset at the root node.

Step 2: For every feature, evaluate every possible split threshold.

Step 3: Compute the impurity measure for each split:

```
Gini Impurity = 1 - Σ(p_i)² [range: 0 to 0.5]
Entropy = -Σ p_i × log2(p_i) [range: 0 to 1]
Information Gain = Entropy(parent) - Σ(|child|/|parent|) × Entropy(child)
```

Step 4: Choose the feature + threshold that gives the HIGHEST Information Gain (or lowest weighted Gini).

Step 5: Split the data on that feature. Create child nodes.

Step 6: Recursively repeat Steps 2–5 on each child node.

Step 7: Stop when:

• Node is pure (all same class) OR

• Max depth reached OR

• Min samples per leaf reached OR

• No further improvement possible

Step 8: Assign the majority class (classification) or mean value (regression) to each leaf.

Key Hyperparameters

Parameter	Effect
max_depth	Controls tree depth — limits overfitting
min_samples_split	Min samples to split a node (default=2)
min_samples_leaf	Min samples in each leaf
criterion	'gini' or 'entropy' — impurity measure
max_features	Number of features to consider at each split

✔ Root node = feature with highest Information Gain across entire dataset.

✔ Pure node: all samples same class → Entropy=0, Gini=0 → stop splitting.

■ Decision Trees overfit easily → use max_depth, min_samples_leaf, or use ensemble methods.

★ A single tree is highly interpretable. Ensemble methods (RF, XGB) trade interpretability for accuracy.

2. Bagging (Bootstrap Aggregating)

Intuition

Bagging trains many models independently on different random subsets of the data (bootstrap samples = sampling WITH replacement). Final prediction = majority vote (classification) or average (regression). Reduces VARIANCE — makes models more stable.

Algorithm — Step by Step

Step 1: From training set D of size N, create B bootstrap samples:

Each bootstrap sample D_b: draw N samples WITH replacement from D.

(~63% unique samples per bootstrap; ~37% are duplicates = out-of-bag samples)

Step 2: Train a base model M_b on each bootstrap sample D_b independently.

(Models trained in PARALLEL — no dependence between them)

Step 3: Aggregate predictions:

Classification → Majority Vote among M_1, M_2, ..., M_B

Regression → Average of M_1(x), M_2(x), ..., M_B(x)

```
Final Prediction (classification) = mode{ M_1(x), M_2(x), ..., M_B(x) }
Final Prediction (regression) = (1/B) × Σ M_b(x)
```

Why does Bagging reduce Variance?

Each model sees different data → makes different errors. When many uncorrelated models vote together, individual errors cancel out. Variance of average = Var(single model) / B (if models independent).

✔ Out-of-Bag (OOB) samples (~37%) can be used as a built-in validation set — no separate val split needed.

■ Bagging does NOT reduce bias. If base model is biased, ensemble will also be biased.

★ Random Forest = Bagging + feature randomness at each split.

3. Random Forest

What is it?

Random Forest = Bagging + Feature Randomness. It builds many Decision Trees on bootstrap samples, but at EACH SPLIT, only a random subset of features is considered. This de-correlates the trees, reducing variance further compared to plain Bagging.

Algorithm — Step by Step

Step 1: Set number of trees B (n_estimators), and feature subset size m.

Classification: m = √(total features) by default

Regression: m = total_features / 3 by default

Step 2: For each tree b = 1 to B:

a) Create bootstrap sample D_b (N samples with replacement from D).

b) Grow a Decision Tree on D_b with the following modification:

• At each node split: randomly select m features (not all features).

• Find best split among ONLY those m features.

• This is the KEY difference from plain Bagging of Decision Trees.

c) Grow tree to maximum depth (no pruning — each tree overfits intentionally).

Step 3: Aggregate: majority vote (classification) or mean (regression).

Why Feature Randomness?

Without feature randomness, if one feature is very strong, ALL trees would split on it first → trees are highly correlated → variance reduction is small. By randomly selecting m features, trees are decorrelated → ensemble is stronger.

Key Hyperparameters

Parameter	What it Controls
n_estimators	Number of trees — more = better but slower (typical: 100–500)
max_features	Features per split — 'sqrt' for clf, 'log2', integer, or float
max_depth	Max depth of each tree (None = fully grown)
min_samples_split	Min samples to split an internal node
min_samples_leaf	Min samples in a leaf node
bootstrap	True = use bootstrap samples; False = use entire dataset
oob_score	True = compute out-of-bag score as validation estimate
class_weight	'balanced' = adjust weights for imbalanced classes

Parametric or Non-Parametric?

Random Forest is NON-PARAMETRIC. It does not assume a fixed functional form of the data. The 'parameters' of a Decision Tree (splits, thresholds) are not learned by gradient descent — they are

determined by the data structure. Each tree can grow arbitrarily complex.

✔ RF is robust to outliers, handles missing values, gives feature importances automatically.

✔ OOB score ≈ cross-validation score — useful when data is small.

■ RF can overfit with very noisy data or too deep trees. Use min_samples_leaf to control.

4. Boosting

Intuition

Boosting trains models SEQUENTIALLY. Each model focuses more on samples the previous model got wrong. Models are added one at a time, each correcting the errors of the ensemble so far. Reduces BIAS (and variance). Examples: AdaBoost, Gradient Boosting, XGBoost, LightGBM.

AdaBoost — Algorithm Step by Step

Step 1: Assign equal weight w_i = 1/N to each training sample i.

Step 2: For t = 1 to T (number of weak learners):

a) Train a weak learner h_t on data with current sample weights.

(Weak learner = simple model, e.g., Decision Tree stump with max_depth=1)

b) Compute weighted error:

```
ε_t = Σ w_i × I(y_i ≠ h_t(x_i)) [sum of weights of misclassified samples]
```

c) Compute learner weight (how much to trust this model):

```
α_t = 0.5 × ln((1 - ε_t) / ε_t)
```

d) Update sample weights — INCREASE weight of misclassified, decrease of correct:

```
w_i ← w_i × exp(α_t × I(y_i ≠ h_t(x_i))) then normalise so Σw_i = 1
```

Step 3: Final prediction = weighted majority vote:

```
H(x) = sign( Σ α_t × h_t(x) )
```

✔ Models with lower error get higher α_t (more say in final vote).

■ AdaBoost is sensitive to noisy data and outliers — they get very high weights.

5. Gradient Descent

What is it?

Gradient Descent is an optimisation algorithm used to minimise a loss function by iteratively moving parameters in the direction of steepest descent (negative gradient).

Algorithm — Step by Step

Step 1: Initialise model parameters θ randomly (or zeros).

Step 2: Compute the Loss Function L(θ) on training data.

Example: MSE = (1/N) × Σ(y_i - ■_i)²

Step 3: Compute the gradient (partial derivatives) of L w.r.t. each parameter:

```
∂L/∂θ_j = gradient telling which direction increases the loss
```

Step 4: Update each parameter in the OPPOSITE direction of gradient:

```
θ_j ← θ_j - η × (∂L/∂θ_j)
```

where η (eta) = learning rate (step size)

Step 5: Repeat Steps 2–4 until:

• Loss converges (change < threshold) OR

• Max iterations (epochs) reached

Variants of Gradient Descent

Variant	Data Used per Update	Pros	Cons
Batch GD	Entire dataset	Stable convergence	Slow for large data
Stochastic GD (SGD)	1 sample	Fast, online learning	Noisy, unstable
Mini-Batch GD	Small batch (32–256)	Balance of speed & stability	Batch size tuning needed

Learning Rate η — Effect

η too HIGH → Overshoots minimum → Loss oscillates or diverges

η too LOW → Very slow convergence → Many iterations needed

η just right → Smooth convergence to minimum

✔ Adaptive methods (Adam, RMSProp, Adagrad) automatically adjust η per parameter.

★ In boosting, learning_rate shrinks each tree's contribution — smaller = more trees needed but better generalisation.

6. Gradient Boosting

Intuition

Gradient Boosting fits new trees on the RESIDUAL ERRORS (pseudo-residuals) of the current ensemble. It is gradient descent in function space — each tree is a 'step' that moves the predictions closer to the truth.

Algorithm — Step by Step

Step 1: Initialise the model with a constant prediction (e.g., mean of y for regression):

```
F_0(x) = argmin_γ Σ L(y_i, γ) → For MSE: F_0(x) = mean(y)
```

Step 2: For m = 1 to M (number of trees):

a) Compute PSEUDO-RESIDUALS (negative gradient of loss at current prediction):

```
r_im = -[∂L(y_i, F(x_i)) / ∂F(x_i)] evaluated at F = F_{m-1}
```

For MSE: r_im = y_i - F_{m-1}(x_i) [actual - predicted = residual]

b) Fit a Decision Tree h_m to the pseudo-residuals r_{im}.

c) Find optimal step size γ_m (line search):

```
γ_m = argmin_γ Σ L(y_i, F_{m-1}(x_i) + γ × h_m(x_i))
```

d) Update the ensemble:

```
F_m(x) = F_{m-1}(x) + η × γ_m × h_m(x)
```

η = learning_rate (shrinks each tree's contribution to prevent overfitting)

Step 3: Final model: F_M(x) = F_0(x) + η × Σ_{m=1}^{M} h_m(x)

Why 'Gradient' Boosting?

Traditional Boosting (AdaBoost) reweights samples. Gradient Boosting generalises this: instead of reweighting, it fits a tree to the NEGATIVE GRADIENT of any differentiable loss function. By changing the loss function, you get different algorithms: MSE → regression, log-loss → classification.

✔ Each tree corrects the mistakes of the ensemble built so far — sequentially.

✔ Smaller learning_rate + more trees = better generalisation (but slower training).

■ n_estimators too high with high learning_rate → overfitting. Use early stopping.

7. XGBoost (Extreme Gradient Boosting)

What Makes XGBoost Special?

XGBoost is an optimised, regularised implementation of Gradient Boosting. It adds L1 + L2 regularisation to the loss, uses second-order Taylor expansion for smarter splits, and is engineered for speed (parallel split-finding, cache awareness, out-of-core computation).

Algorithm — Step by Step

Step 1: Initialise with a base prediction (e.g., 0.5 for binary classification).

Step 2: For each tree m = 1 to M:

a) Compute first-order gradient (g_i) and second-order gradient (h_i):

```
g_i = ∂L(y_i, ■_i) / ∂■_i [first derivative — direction]
h_i = ∂²L(y_i, ■_i) / ∂■_i² [second derivative — curvature]
```

b) Find best split using Gain formula (XGBoost's unique scoring):

```
Gain = 0.5×[(Σg_L)²/(Σh_L+λ) + (Σg_R)²/(Σh_R+λ) - (Σg)²/(Σh+λ)] - γ
```

Split is made only if Gain > 0 (γ is min gain threshold = regularisation)

c) Compute optimal leaf weights:

```
w_j* = -(Σ g_i in leaf j) / (Σ h_i in leaf j + λ)
```

d) Update predictions: ■_i ← ■_i + η × w_{leaf(x_i)}

Step 3: Regularised Objective Function (what XGBoost minimises):

```
Obj = Σ L(y_i, ■_i) + Σ_m Ω(f_m)
Ω(f) = γ×T + 0.5×λ×Σw_j² [T=num leaves, λ=L2 reg, γ=min gain]
```

Key Hyperparameters

Parameter	What it does
n_estimators	Number of trees
learning_rate (eta)	Shrinkage per tree — smaller = more trees needed
max_depth	Max depth per tree (typically 3–10)
subsample	Fraction of samples per tree (e.g., 0.8) — reduces overfitting
colsample_bytree	Fraction of features per tree — like RF's max_features
colsample_bylevel	Fraction of features per level/depth
lambda (reg_lambda)	L2 regularisation on leaf weights
alpha (reg_alpha)	L1 regularisation on leaf weights
gamma (min_split_loss)	Min gain to make a split — higher = more conservative
min_child_weight	Min sum of h_i in a leaf — prevents over-small leaves
eval_metric	Metric to evaluate during training (logloss, rmse, auc, etc.)
early_stopping_rounds	Stop if val metric doesn't improve for N rounds

✔ If model overfits: decrease max_depth, decrease n_estimators or use early stopping, increase lambda.

✔ If model underfits: increase max_depth, increase n_estimators, increase learning_rate.

★ XGBoost grows trees LEVEL-WISE (breadth-first). LightGBM grows LEAF-WISE — key architectural difference.

8. LightGBM (Light Gradient Boosting Machine)

What Makes LightGBM Different from XGBoost?

Feature	XGBoost	LightGBM
Tree growth	Level-wise (breadth-first)	Leaf-wise (best-first)
Speed	Slower on large data	Much faster (GOSS + EFB)
Memory	Higher	Lower
Overfitting risk	Lower (level-wise more conservative)	Higher (leaf-wise grows deeper)
Categorical handling	Requires encoding	Native categorical support
Large datasets	Good	Excellent

Leaf-Wise Tree Growth

Unlike XGBoost (which grows all nodes at a given depth before going deeper), LightGBM always splits the leaf with the MAXIMUM LOSS REDUCTION — regardless of depth. This means trees can be asymmetric and grow very deep on one branch, capturing complex patterns faster.

Two Key Innovations

1. GOSS — Gradient-based One-Side Sampling

- Keep ALL samples with LARGE gradients (they are the hard/important ones).
- RANDOMLY SAMPLE a fraction of samples with small gradients.
- This massively reduces data size while preserving accuracy.
- Mathematically compensates small-gradient samples by a constant amplification factor.

2. EFB — Exclusive Feature Bundling

- High-dimensional sparse data often has mutually exclusive features

(i.e., features that rarely take non-zero values simultaneously).

- EFB bundles such features into a single feature — reduces #features dramatically.
- This speeds up split-finding with negligible information loss.

Key Hyperparameters

Parameter	What it does
n_estimators	Number of trees/boosting rounds
learning_rate	Shrinkage factor (smaller = need more trees)
num_leaves	Max leaves per tree — KEY param (controls complexity, risk of overfit)
max_depth	Limits depth (–1 = no limit; use with num_leaves for control)
min_child_samples	Min samples per leaf (prevents overfitting)
subsample (bagging_fraction)	Fraction of data per tree
colsample_bytree (feature_fraction)	Fraction of features per tree
lambda_l1 / lambda_l2	L1 / L2 regularisation

min_split_gain	Min gain to split — acts like XGBoost's gamma
class_weight / is_unbalance	Handle class imbalance

✔ 'Light' in LightGBM refers to: faster training, lower memory, not a lighter/weaker model.

■ num_leaves has more impact than max_depth in LGBM. Start with num_leaves = 2^max_depth - 1.

9. Logistic Regression

What is it?

Logistic Regression is a linear classification model that predicts the PROBABILITY of class membership using the sigmoid (logistic) function. Despite the name 'regression', it is used for classification.

Algorithm — Step by Step

Step 1: Initialise weights w and bias b (usually to 0 or random small values).

Step 2: Compute the linear combination (logit):

```
z = w · x + b = w_1×x_1 + w_2×x_2 + ... + w_n×x_n + b
```

Step 3: Apply the sigmoid function to get probability:

```
σ(z) = 1 / (1 + e^(-z)) output ∈ (0, 1)
```

Step 4: Predict class:

```
■ = 1 if σ(z) ≥ 0.5 else 0 [threshold adjustable]
```

Step 5: Compute Binary Cross-Entropy Loss (Log Loss):

```
L = -(1/N) × Σ [y_i × log(■_i) + (1-y_i) × log(1-■_i)]
```

Step 6: Compute gradient of loss w.r.t. weights:

```
∂L/∂w_j = (1/N) × Σ (■_i - y_i) × x_ij
```

Step 7: Update weights using Gradient Descent:

```
w ← w - η × ∂L/∂w
```

Step 8: Repeat Steps 2–7 until convergence.

Why Log Loss and not MSE?

MSE with sigmoid output creates a non-convex loss surface → gradient descent gets stuck in local minima. Log Loss is CONVEX for logistic regression → guaranteed convergence to global minimum.

Regularisation

L2 (Ridge): adds λ × Σw_j² to loss → shrinks all weights, prevents overfitting

L1 (Lasso): adds λ × Σ|w_j| to loss → drives some weights to EXACTLY 0 → feature selection

Elastic Net: combination of L1 + L2

```
sklearn param 'C' = 1/λ → smaller C = stronger regularisation
```

Multi-class Extension

One-vs-Rest (OvR): Train K binary classifiers. Predict class with highest probability.

Softmax (Multinomial): Single model with softmax output → directly predicts K-class probabilities.

```
softmax(z_k) = e^z_k / Σ e^z_j for k = 1...K
```

✔ Logistic Regression is a PARAMETRIC, LINEAR, DISCRIMINATIVE model.

■ Logistic Regression assumes LINEAR decision boundary → fails for non-linear problems without feature engineering.

★ Why called 'Regression'? Because it models a continuous output (probability) and fits a line — but uses a threshold for classification.

10. KMeans Clustering

What is it?

KMeans is an unsupervised clustering algorithm that partitions N data points into K clusters by minimising the Within-Cluster Sum of Squares (WCSS / Inertia).

Objective Function

```
WCSS = Σ_{k=1}^{K} Σ_{x_i ∈ C_k} ||x_i - μ_k||²
```

where μ_k is the centroid (mean) of cluster k. Goal: minimise WCSS.

Algorithm — Step by Step (Lloyd's Algorithm)

Step 1: Choose K (number of clusters).

Step 2: INITIALISE K centroids randomly from the data points (or random positions).

Step 3: ASSIGNMENT step — assign each point x_i to nearest centroid:

```
c_i = argmin_k ||x_i - μ_k||²
```

Step 4: UPDATE step — recompute each centroid as the mean of assigned points:

```
μ_k = (1/|C_k|) × Σ_{x_i ∈ C_k} x_i
```

Step 5: Repeat Steps 3–4 until:

• Centroids do not change (convergence) OR

• Max iterations reached

Step 6: Final clusters = current assignments.

How to Choose K? — Elbow Method

1. Run KMeans for K = 1, 2, 3, ..., 10
2. Plot K vs. WCSS (Inertia)
3. WCSS decreases as K increases (adding more clusters reduces compactness)
4. The 'elbow' in the graph — where WCSS starts decreasing slowly — is the optimal K

✔ Silhouette score is another metric: measures how well separated clusters are. Range: -1 to +1, higher is better.

Limitations of KMeans

• Must specify K in advance

• Sensitive to random initialisation (may converge to local minimum)

• Assumes spherical, equal-sized clusters

• Sensitive to outliers (outlier pulls centroid)

• Sensitive to feature scale → ALWAYS scale features before KMeans

11. KMeans++ (Smart Initialisation)

Why KMeans++?

Plain KMeans randomly initialises centroids — this can lead to bad starting points → slow convergence or suboptimal clusters. KMeans++ uses a smarter initialisation: spread initial centroids far apart, so they're more likely to end up in different clusters from the start.

Initialisation Algorithm — Step by Step

Step 1: Choose the FIRST centroid μ_1 uniformly at random from all data points.

Step 2: For each remaining centroid k = 2, 3, ..., K:

a) For each data point x_i, compute D(x_i) = distance to nearest already-chosen centroid:

```
D(x_i) = min_{j < k} ||x_i - μ_j||²
```

b) Choose next centroid μ_k with probability PROPORTIONAL to D(x_i)²:

```
P(x_i selected) = D(x_i)² / Σ_j D(x_j)²
```

c) Points FAR from existing centroids have HIGHER probability of being selected.

Step 3: After all K centroids are initialised, proceed with standard KMeans (Steps 3–5 above).

Why D(x_i)² (squared distance)?

Using squared distance gives even more weight to far-away points. This ensures centroids are well spread across the data space, reducing the chance of two centroids initialised in the same cluster.

✔ KMeans++ is the DEFAULT initialisation in sklearn's KMeans (init='k-means++').

✔ KMeans++ guarantees that the final WCSS is at most O(log K) times the optimal WCSS.

★ KMeans++ typically requires fewer iterations to converge AND achieves lower final inertia than random initialisation.

12. Quick Comparison — All Algorithms

Algorithm	Type	Parallel?	Parametric?	Reduces	Best For
Decision Tree	Supervised	N/A	No	—	Interpretability
Bagging	Ensemble	Yes	Depends	Variance	Stable models
Random Forest	Ensemble	Yes	No	Variance	General classification/regression
AdaBoost	Ensemble	No (seq)	No	Bias	Low-complexity tasks
Gradient Boosting	Ensemble	No (seq)	No	Bias+Var	Tabular data
XGBoost	Ensemble	Partial	No	Bias+Var	Competitions, structured data
LightGBM	Ensemble	Partial	No	Bias+Var	Large datasets, speed
Logistic Regression	Linear	N/A	Yes	—	Binary classification baseline
KMeans	Clustering	N/A	No	—	Clustering, customer segmentation
KMeans++	Clustering	N/A	No	—	Better KMeans initialisation
Gradient Descent	Optimiser	N/A	N/A	—	Training all parametric models

---

All the best for your viva! Know your notebook well, understand WHY you used each technique, and be ready to explain any algorithm from first principles.